

```
# Test with 5 per page, page #2
url =
➡ "/bands/musicians/?items_per_page=5&page=2"
response = self.client.get(url)

self.assertEqual(200, response.status_code)
self.assertEqual(5,
    len(response.context["musicians"]))
self.assertTrue(response.context["page"].has_previous())
self.assertFalse(response.context["page"].has_next())
```

Set both the number of items on the page as well as the page number.

Validate the right number of musicians came back and there is a previous page but no next page.

Good tests check for error conditions as well. The following listing checks that a bad value for the number of items per page is handled properly.

Listing E.38 Handling bad query parameter values as well

```
# RiffMates/bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_musicians(self):
        ...
        # Test bad page size
        url = "/bands/musicians/?items_per_page=-1"
        response = self.client.get(url)

        self.assertEqual(200, response.status_code)
        self.assertEqual(10, len(response.context["musicians"]))
        self.assertFalse(response.context["page"].has_other_pages())
```

Negative numbers for items_per_page are not allowed.

Bad values result in the default of 10 musicians per page.

E.8 Chapter 9

Sometimes, a web-based interface isn't the best choice, especially for dealing with managerial tasks for your site. Chapter 9 covers how to write your own Django management command.

Exercise 1

Create a `venues` management command that lists all the Venues in your database. Include a `--rooms` flag to optionally display the names of the rooms associated with a venue. Hint: the `action="store_true"` parameter to `add_arguments` changes your flag, so it acts as a Boolean flag and doesn't expect any additional arguments.

The following listing shows the management command to list venues and, optionally, list their associated rooms.

Listing E.39 Management command to list venues

```
# RiffMates/bands/management/commands/venues.py
from bands.models import Venue
```